1 Name of Use Case

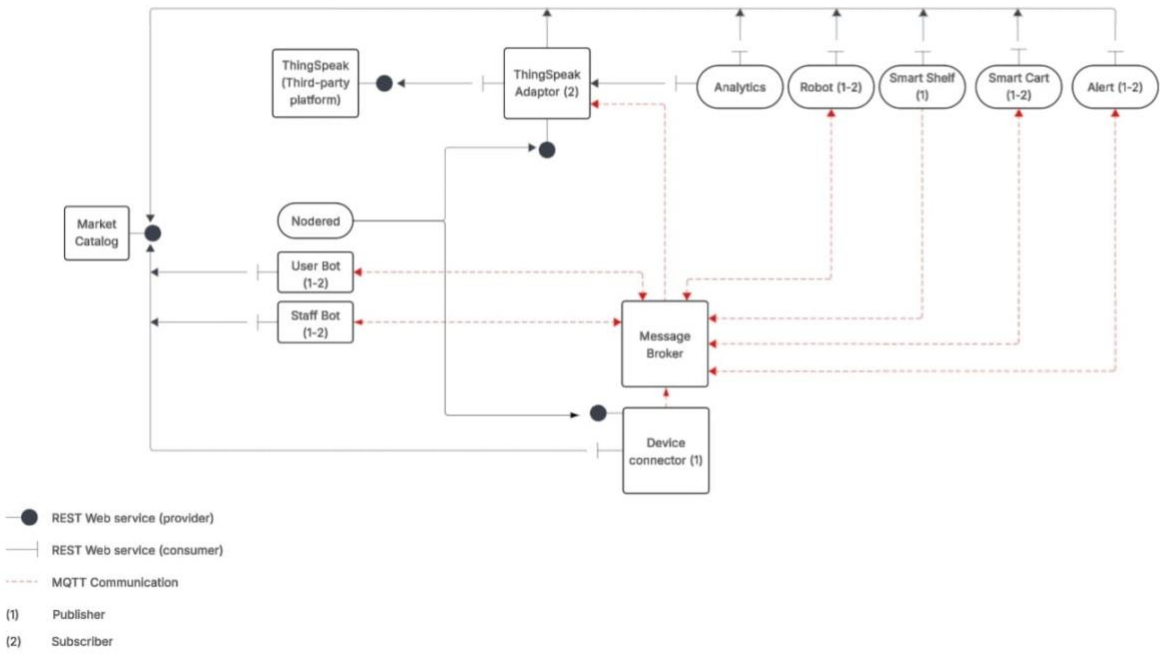
Name of the Use Case	**IoT platform for Smart Market**
Version No.	v0.2
Date	27/11/2025
Team Members (with student ids)	Trippetta Simone (S360724), Oliveri Luca (S358011), Fabrizio D'Amico (S360457), Andrea Lisi (S345954)

2 Scope and Objectives of Function

Scope and Objectives of Use Case	
Scope	The proposed IoT platform aims to digitalize and automate a supermarket through a microservices architectur. The system integrates smart shopping carts, real-time inventory management with autonomous robots, sensor for monitoring environmental conditions and Telegram bots for both customers and store staff.
Objective(s)	The primary objectives are to automate product management within smart shopping carts and smart shelves, streamline the checkout process, and provide store staff with real-time control over operations. Additionally, the platform monitors customer behavior (dwell time), sales trends, and environmental anomalies. Inventory updates are managed autonomously by Smart Shelves and Robots to prevent stockouts.
Domain(s)	Smart Building, Smart Retail.
Stakeholder(s)	Supermarket customers, Store staff, Warehouse managers, Store management and administration, Cashier.
Short description	The Smart Market platform automates the retail experience by integrating RFID-enabled Smart Carts that synchronize directly with a User Telegram Bot, enabling real-time shopping list updates and generating code for the checkout. On the operational side, **Smart Shelves** automatically detect low stock and trigger **Autonomous Robots** to retrieve products from the warehouse while a **Staff Bot** allows employees to manage alerts and finalize transactions. The ecosystem is completed by smart sensors for monitoring environmental conditions (temperature, crowd levels) and an analytics module for tracking sales trends and customer dwell time, all supported by a microservices architecture using REST and MQTT protocols. **Key Features Summary:** • Smart Cart with RFID technology connected to the User Bot. • Staff Telegram Bot for checkout operations and alert management. • Automatic low-stock alerting and real-time database updates sent by Smart Shelves and processed by Autonomous Robots. • Environmental sensors visualizable by both customers and staff. • Communication is based on REST and MQTT protocols.

3 Diagram of Use Case

CART BOT?



4 Complete description of the system

The proposed IoT platform for Smart Market follows the microservices designing pattern. It also exploits two communication paradigms: i) publish/subscribe based on MQTT protocol and ii) request/response based on REST Web Services.

In this context, 13 actors have been identified and introduced in the following:

- The **Message Broker** provides asynchronous communication based on the publish/subscribe approach. It exploits the MQTT protocol.

- The **Market Catalog** works both as device and service registry for the entire system.
  It acts as a REST provider, exposing Web Services queried by all other actors to retrieve heterogeneous information.

- The **Device Connector (Raspberry Pi Connector)** integrates the platform Raspberry Pi boards. It collects environmental data (temperature and humidity) from the market and fridge areas. We will use two Raspberry Pi connector: one for the fridge and one for the market environment.
  Connections:
  - Acts as a **REST Consumer** to register itself with the Market Catalog.
  - Acts as an **MQTT Publisher** to send real-time environmental data to the Message Broker.
  - Acts as a **REST Provider** to expose live data directly to the Node-RED dashboard.

- The **Thingspeak Adaptor** receives raw measurements and analytics reports, uploading them to the ThingSpeak cloud platform.

  Maybe analytics stored somewhere else? Thingspeak should only store time-series data

  Connections:
  - Acts as an **MQTT Subscriber** to receive data from the Broker.
  - Acts as a **REST Consumer** to push data to ThingSpeak (Third-party platform).

  - Acts as a **REST Provider** to expose historical time-series data to the Node-RED dashboard.
- **Thingspeak** is a third-party software (https://thingspeak.com/) that provides REST Web Services. It is an open-data platform for the Internet of Things to store and visualize data (through plots).

- **Nodered** is a dashboard to retrieve data and visualize them. It retrieves both historical and live data. Connections:
  - Acts as a **REST Consumer** to pull historical data from the ThingSpeak Adaptor.
  - Acts as a **REST Consumer** to pull live environmental data from the Device Connector.

- The **Smart Cart** is composed by an **RFID sensor** and a smart touch screen (**Cart Telegram Bot**). It detects items placed in the cart and updates the virtual shopping session.
  Connections:
  - RFID Sensor: Acts as an MQTT Publisher, sending the detected Item IDs to the Broker.
  - Cart Bot: Acts as an MQTT Subscriber to receive the wish list from the User Bot and updates from the RFID sensor on which product was added to the cart.
    Acts as a REST Consumer to query product details (name, price) from the Catalog.
    Acts as MQTT subscriber to publish the payment code to the Broker.

- **User Telegram Bot** is the customer interface for catalogue browsing and checkout.
  Allows users to create wishlists, pair with a Smart Cart, visualize the Nodered dashboard and process payments.
  Connections:
  - Acts as a REST Consumer to retrieve the product list from the Catalog.
  - Acts as an MQTT Publisher to send wishlist data to the cart.
  - Acts as an MQTT Subscriber to receive the final payment confirmation and QR code.

- **Staff Telegram Bot** is the management tool for store employees. Allows staff to monitor crowd levels, robot status, visualize the Nodered dashboard and handle alerts.
  Connections:
  - Acts as an MQTT Subscriber to receive alerts (low stock, high temperature) generated by the Alert System.
  - Acts as a REST Consumer to manually update the Catalog (e.g., changing product prices or stock).

- The **Smart Shelf** is an IoT device capable of monitoring product availability in real-time thanks to RFID technology. It performs continuous local polling of the tags present on the shelf. Ideally, it applies a "Report by Exception" logic: the local controller compares the tag list at time T with time T-1.
  Connections:
  - If an ID disappears (removal) or appears (addition), it acts as an MQTT Publisher, sending an event (to the Broker to notify the Alert System.
  - Acts as a REST Consumer to register itself with the Market Catalog.

- The **Robot service** manages the autonomous restocking agents. It waits for specific restocking commands generated by the Alert System when stock is low. It coordinates the robot's movement to retrieve products from the warehouse and refill the shelves.
  Connections:
  - Acts as an **MQTT Subscriber** to receive specific "Restock" commands from the Alert System.
  - Acts as an **MQTT Publisher** to broadcast its status (e.g., "Moving", "Refilling") to the Staff Bot.
  - Acts as a **REST Consumer** to query the Catalog for product warehouse locations.

- The **Alert System** acts as the central operational brain of the supermarket. It verifies real-time data from environmental sensors and processes events from Smart Shelves. It updates the inventory in the Catalog upon receiving RFID events and checks safety thresholds (e.g., fridge temperature, minimum stock levels). If a threshold is breached, it triggers alerts to the Staff Bot or sends mission commands to the Robot Service.
  Connections:
  - Acts as an MQTT Subscriber to receive sensor data (temperature) and shelf events (RFID updates).
  - Acts as a REST Consumer to update stock levels in the Catalog and retrieve product details.
  - Acts as an MQTT Publisher to send alarms to the Staff Bot and restocking commands to the Robot Service.

- **Analytics Service** provides a historical view of the market. It processes transaction data to calculate crowd levels, shopping habits, and sales history.
  Connections:
  - Acts as a **REST Consumer** to retrieve transaction logs from the Catalog.
  - Acts as a **REST Consumer** to push the processed reports directly to the ThingSpeak Adaptor via REST APIs for visualization.

Reports shouldnt be saved in Thingspeak

REST consumer of Thingspeak Adaptor? To retrieve historical data and perform the analysis

Desired Hardware components (only among those we can provide)

Device Name	Quantity	Needed for…
Raspberry	1	

- Do you really need to separate "User Bot" and "Cart Bot"? Maybe the same bot can handle the functionalities. Try to analyse if it is worthy splitting it into two bots or only one. Moreover, Cart bot is missing in the use case diagram.
- Do you plan to make a "smart" routing of the robot? that might not be straightforward
- Be aware that the Analytics must be complex enough as pre-processing service, employing the historical data from thingspeak to perform some calculations.